实验课程名称：__软件工程基础实验_____

实验项目名称	Mini-12306 静态代码审查与性能优化			实验成绩	
实 验 者		专业班级		组　　别	
同 组 者				实验日期	2026-10-22

第一部分：实验预习报告（包括实验目的、意义，实验基本原理与方法，主要仪器设备及耗材，实验方案与技术路线等）

1.1 实验目的与意义

本实验以 Mini-12306 购票系统为对象，完成两个相互独立的子实验：静态代码审查与性能优化。

- 静态代码审查：掌握 ESLint、Semgrep、npm audit 三类工具的选择依据、执行方法和结果判读；能够结合数据流区分真实风险与启发式误报；在“只审查、不改代码”的约束下形成可复核的审查结论。
- 性能优化：掌握“基线测量—瓶颈定位—代码优化—复测验证”的完整闭环；使用 Clinic Flame 定位 CPU 热点、Autocannon 量化吞吐与延迟、MongoDB profiler 验证执行计划与扫描量；能够区分数据库层指标与端到端指标。
- 工程素养：保留原始数据、优化前后源码快照、diff 与复现命令，使实验结论可重复；对含有真实凭据的配置保持隔离，不因实验读取或泄露敏感信息。

实验意义在于：静态工具能以较低成本暴露规范、安全和依赖问题，但工具告警并不等于漏洞，人工复核不可省略；性能优化不能凭直觉改代码，必须先测量，并且要认识到局部数据库指标的大幅改善会被整条请求链路的其他开销稀释。

1.2 实验基本原理与方法

1.2.1 静态代码审查原理

- ESLint：把 JavaScript 解析为 AST，再按规则匹配语法结构，可发现未使用变量、块内函数声明、不可见空白等问题；eslint-plugin-security 通过启发式模式检查动态对象注入、eval、不安全正则等风险。优点是本地运行、可配置、可导出 HTML；缺点是安全规则只做模式判断，误报率较高。
- Semgrep：使用语义模式规则跨行匹配代码，本次启用 p/nodejs 与 p/owasp-top-ten 规则集，重点检查 XSS、命令注入和不安全反序列化。它能表达比 ESLint 更贴近数据流的模式，但仍需人工确认实际数据是否被转义。
- npm audit：把依赖树与官方漏洞库比对，报告已知 CVE。它只覆盖第三方依赖，不检查业务代码，因此必须与代码级工具配合使用。

1.2.2 性能分析原理

- Clinic Flame：基于 V8 采样生成火焰图，按调用栈聚合样本，用于定位 CPU 热点。采样会带来额外开销，因此它适合“找热点”，不适合作为绝对吞吐基准。
- Autocannon：固定并发连接和持续时间发送 HTTP 请求，输出吞吐量、平均延迟及 P50/P97.5/P99 等分位数。通过固定查询、固定数据集和多轮重复，降低单次测量波动。
- MongoDB profiler：将级别设为 2 后，数据库把每次操作写入 system.profile，可读取 planSummary、docsExamined、keysExamined、nReturned 和 executionTimeMillis，用来判断查询是全表扫描还是索引扫描。
- 复合索引与多路扫描：MongoDB 使用 B-tree 索引。对“出发城市/车站 × 到达城市/车站”四种等值组合分别建立复合索引，使每个 $or 分支都能独立命中索引，再由查询计划合并结果，避免 COLLSCAN。operatingDates 是数组字段，会建立 multikey 索引，对数组元素做等值匹配仍可走索引。
- 复杂度优化：原代码对每个车次的每个席别都在库存数组中线性查找，复杂度为 O(车次数 × 席别数 × 库存数)；先把“车次|席别”建成 Map，可将查找降为 O(1)，整体降为 O(库存数 + 车次数 × 席别数)。
- JWT 密钥对象：jsonwebtoken 在传入字符串密钥时，每次 verify 都会先尝试 createPublicKey，失败后再回退到 HMAC。把字符串在启动时转换为 KeyObject，可跳过每个请求上的失败解析。

1.3 主要仪器设备及耗材

类别	名称及版本	用途
运行平台	Windows 11 x64（build 26300），Intel Core Ultra 5 125H，24 GB 内存	实验运行环境
运行时	Node.js 24.18.0、npm 11.16.0	运行服务、测试与工具
数据库	MongoDB 7.0.14（Docker Desktop 29.9.2 / Linux 容器，官方 mongo:7.0.14 镜像，单节点副本集 rs0，127.0.0.1:27017）	隔离测试库 mini12306_perf
浏览器	Microsoft Edge + Puppeteer-core 24	将 HTML 报告渲染为截图
静态审查	ESLint 8.57.1 + eslint-plugin-security 3.0.1；Semgrep 1.182.0；npm audit（npm 11.16.0）	规范、语义安全、依赖漏洞检查
性能分析	Clinic.js 13.0.0（Clinic Flame）；Autocannon 8.x；MongoDB profiler level 2	热点、吞吐/延迟、执行计划与扫描量
被测项目	Mini-12306（原生 HTML/CSS/JS + Node.js 20+/Express 4 + Mongoose）	实验对象
实验数据	8,000 趟车、168,000 条库存、1 个测试用户	放大查询成本，便于观察索引收益

表 1-1 主要仪器设备、软件与实验数据

1.4 实验方案与技术路线

实验一（静态代码审查）的技术路线：

- 确定审查范围：前端 js/**/*.js，后端 server/src/**/*.js；分别采用浏览器与 Node 环境配置。
- 选择互补工具：ESLint 负责规范与常见危险模式，Semgrep 负责语义安全规则，npm audit 负责依赖漏洞。
- 执行并保留原始输出：JSON、文本和 HTML 报告全部落盘，再用无头浏览器截图，避免只保留“结论”而丢失证据。
- 人工复核：对每条安全告警检查数据来源、转义方式和调用路径，明确“真实风险 / 误报 / 待确认”。
- 输出结论：按严重程度和可维护性分类，提出后续清理建议；本实验不修改任何业务代码。

实验二（性能优化）的技术路线：

基线测量 → 瓶颈定位 → 代码优化 → 配对复测 → 正确性验证 → 数据分析

- 隔离环境：用 Docker Desktop 启动独立的 mongo:7.0.14 容器，初始化单节点副本集 rs0，映射到 127.0.0.1:27017；数据库名必须包含 mini12306_perf，测试服务器显式传入测试 JWT 密钥，不读取 server/.env。
- 构造数据：导入 8,000 趟车、168,000 条库存和 1 个测试用户，固定查询“武汉 → 北京，2026-10-25”，命中 3 趟车。
- 采集基线：用 profiler 记录执行计划与扫描量，用 Autocannon 记录 50 连接、3 轮 × 15 s 的吞吐和延迟，用 Clinic Flame 定位 CPU 热点。
- 实施优化：为四种起终点组合增加复合索引并改写查询；用 Map 消除库存线性查找；用 KeyObject 消除 JWT 密钥重复解析。
- 配对复测：在同一份 Docker 数据库上，先恢复未建复合索引的基线快照并压测，再恢复优化代码与索引复测；两阶段使用同一份数据、同一 token、同一压测参数，各 3 轮，减少机器状态漂移的影响。
- 验证与讨论：检查返回车次、余票和 2xx 数量，运行 35 个自动化测试；分别报告数据库层与端到端指标，并用 Amdahl 定律讨论差异。

第二部分：实验过程记录（可加页）（包括实验原始数据记录，实验现象记录，实验过程发现的问题等）

2.1 实验一：静态代码审查过程

2.1.1 审查范围、配置与执行

前端审查范围是 js/**/*.js，使用浏览器环境、ES2022、eslint:recommended 与 eslint-plugin-security；后端审查范围是 server/src/**/*.js，使用 Node 环境与相同规则。配置文件与原始输出均保存在 experiments/static/。

在项目根目录执行的主要命令如下（工具依赖安装在 experiments/tools）：

```
# ESLint（前端）
experiments/tools/node_modules/.bin/eslint \
  -c experiments/static/eslint-browser.json js --ext .js

# ESLint（后端）
experiments/tools/node_modules/.bin/eslint \
  -c experiments/static/eslint-server.json server/src --ext .js

# Semgrep
semgrep --config p/nodejs --config p/owasp-top-ten \
  --json --output experiments/static/semgrep.json .

# npm audit（使用官方 registry）
cd server && npm audit --json --registry=https://registry.npmjs.org
```

过程现象：最初尝试加入 eslint-plugin-no-unsanitized 以检查 innerHTML 的 XSS 风险，但该插件 4.x 要求 ESLint 9，与项目使用的 ESLint 8.57.1 产生 ERESOLVE 依赖冲突。为避免为一次审查升级整套 lint 工具链，最终移除该插件，XSS 风险改由 Semgrep 的 raw-html-concat 规则覆盖；这也是工具选型阶段的一次实际取舍。

2.1.2 ESLint：前端结果

ESLint 前端共报告 24 个问题：15 个 error、9 个 warning，分布在 6 个文件。按文件统计如下：

文件	error	warning	主要规则
js/booking.js	3	1	no-irregular-whitespace、no-inner-declarations、security/detect-object-injection
js/management.js	10	5	no-irregular-whitespace、no-inner-declarations、security/detect-object-injection
js/orders.js	2	0	no-irregular-whitespace
js/home.js	0	1	no-unused-vars

js/role-guard.js	0	1	security/detect-object-injection
js/utils.js	0	1	no-unused-vars
合计	15	9	—

表 2-1　ESLint 前端问题按文件统计

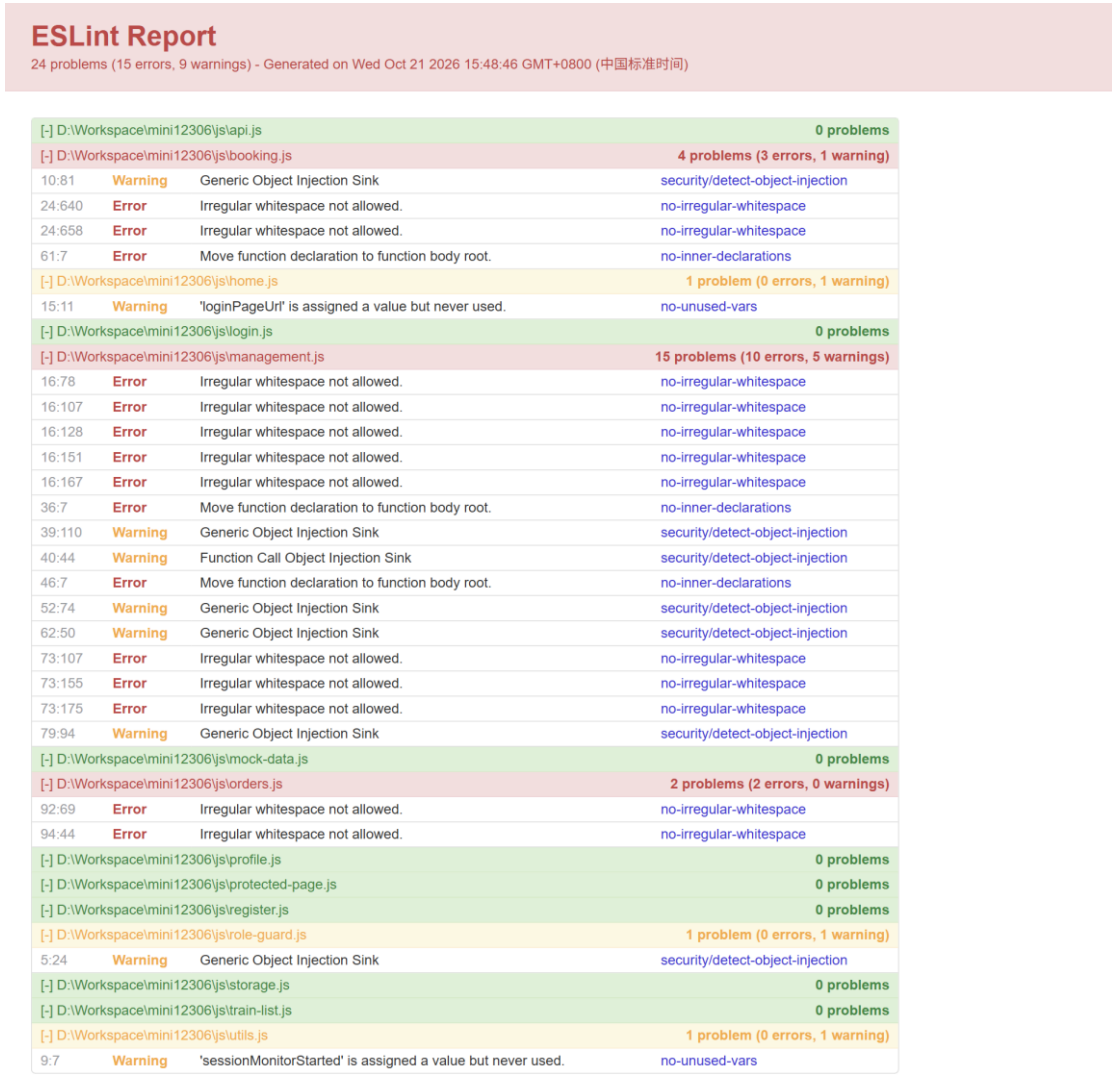


图 2-1　ESLint 前端审查结果（HTML 报告全页截图）

规则	数量	级别	说明与判断
no-irregular-whitespace	12	error	模板字符串中存在不可见空白字符，易造成复制粘贴和字符串比较问题，建议清理。
no-inner-declarations	3	error	在块级作用域内声明函数；现代浏览器可运行，但函数提升语义容易误读。
security/detect-object-injection	7	warning	动态对象下标访问的启发式告警；多数键来自固定枚举或已校验字段，需要逐条确认。

no-unused-vars	2	warning	home.js 的 loginPageUrl、utils.js 的 sessionMonitorStarted 未被使用，属于死代码。

表 2-2　ESLint 前端问题按规则统计

现象分析：15 个 error 中没有会导致运行时崩溃的语法错误，12 个来自不可见空白，3 个来自块内函数声明；7 个对象注入告警属于启发式规则，不能直接判为漏洞。因此前端没有“阻断发布”的问题，但有明确的格式与可维护性清理项。

2.1.3　ESLint：后端结果



图 2-2　ESLint 后端审查结果（HTML 报告全页截图）

后端共报告 15 个 warning，0 个 error。按文件统计如下：

文件	warning	说明
server/src/utils/validation.js	11	security/detect-object-injection；按字段名动态读取校验规则，需要确认字段名来自代码常量或白名单。
server/src/seed.js	2	解构出的 phoneMasked、realNameMasked 未使用。
server/src/utils/order-service.js	1	引入的 RefundRecord 未使用。

server/src/utils/serialize.js	1	解构出的 __v 未使用，用于从输出中剔除版本字段。
合计	15	—

表 2-3　ESLint 后端问题按文件统计

现象分析：后端没有语法或潜在运行时错误。对象注入告警集中在校验工具，属于需要结合调用方确认的启发式规则；未使用变量属于清理项，不影响功能。

2.1.4　Semgrep：语义安全扫描



图 2-3　Semgrep 语义安全扫描结果

Semgrep 共报告 4 条告警，全部是 javascript.browser.security.raw-html-concat.raw-html-concat，严重级别为 WARNING：

文件:行	规则	告警内容
js/booking.js:24	raw-html-concat	用户可控数据进入 HTML 字符串，可能导致 XSS
js/booking.js:48	raw-html-concat	同上

js/booking.js:53	raw-html-concat	同上
js/booking.js:57	raw-html-concat	同上

表 2-4　Semgrep 告警清单

人工复核：js/booking.js 定义了 escape()，对 & < > " ' 做实体编码；routeCard() 中所有订单字段都经过 escape()。第 24/48/53/57 行的 innerHTML 拼接中，插值数据也经过 escape() 或 money()（数值格式化）。因此这 4 条是误报，当前代码路径不构成可利用的 XSS。

但仍需指出：innerHTML + 字符串模板是容易回归的写法，后续新增字段若忘记转义会重新引入风险。更稳妥的长期方案是改用 DOM API / textContent 或 Trusted Types；本实验按要求只记录建议，不修改代码。

2.1.5　npm audit：依赖漏洞

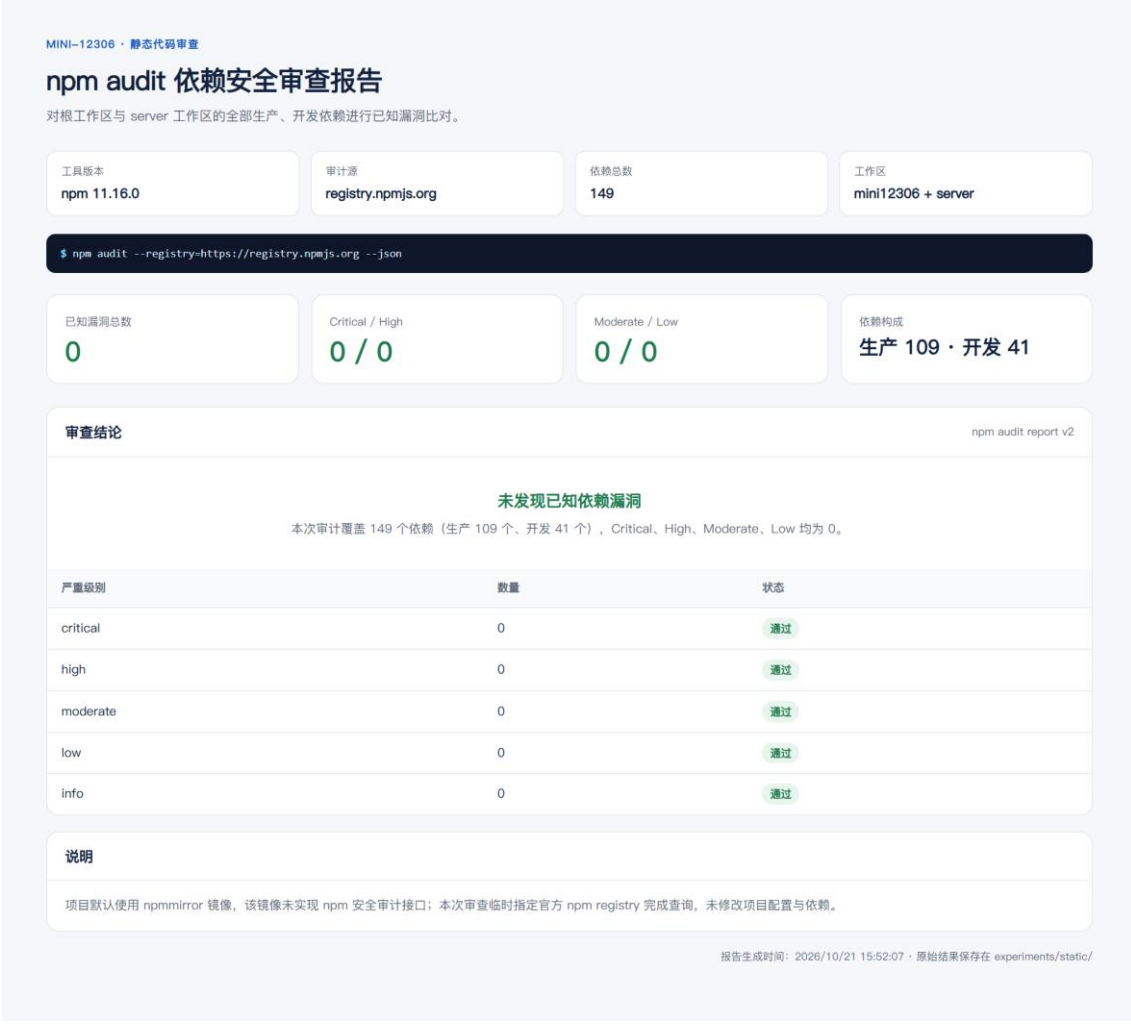


图 2-4　npm audit 依赖漏洞检查结果

过程现象：本机默认使用 npmmirror 镜像时，npm audit 请求 /-/npm/v1/security/advisories/bulk 返回 404，无法得到审计结果。改用官方 registry 后成功，命令为 npm audit --json --registry=https://registry.npmjs.org。

结果：info/low/moderate/high/critical 均为 0，共 0 个漏洞；npm 元数据统计 149 个依赖（prod 109、dev 41，npm 的 total 与分项相加存在 1 个计数口径差异）。该结果只代表依赖的已知 CVE 为 0，不能替代业务代码审查。

2.2 实验二：性能基线测量

2.2.1 优化对象与数据规模

优化对象是车次查询接口 GET /api/trains?from=武汉&to=北京&date=2026-10-25。该请求的完整处理链路包括：JWT 验签 → User.findById → Session.findOne → Train.find（本次优化对象）→ Inventory.find → 组装 JSON 并写回响应。

测试数据为 8,000 趟车、168,000 条库存（8,000 × 7 天 × 3 席别）和 1 个测试用户；固定查询命中 3 趟车 G1001–G1003，席别余票分别为商务座 40、一等座 100、二等座 500。数据库为独立的 mini12306_perf，运行在 Docker 容器的单节点副本集 rs0 中（mongo:7.0.14，127.0.0.1:27017）。

测试服务器 experiments/perf/server.js 显式传入测试 JWT 密钥，不读取 server/.env；seed-large.js 只有在数据库名包含 mini12306_perf 时才允许清库导入，避免误操作开发数据。

2.2.2 测量环境与压测参数

项目	配置
CPU 分析	Clinic.js 13.0.0 / Clinic Flame，30 连接、15 s（仅用于热点分析）
负载生成	Autocannon 8.x，50 连接，3 s 预热 + 3 轮 × 15 s
数据库执行计划	MongoDB profiler level 2，读取 system.profile
MongoDB	7.0.14（官方 mongo:7.0.14 镜像，单节点副本集 rs0，127.0.0.1:27017），数据库 mini12306_perf
运行环境	Windows 11 x64（build 26300），Intel Core Ultra 5 125H，24 GB 内存；Node.js 24.18.0；测试服务器监听 3100 端口
配对方式	同一份 Docker 数据库、同一 token、同一查询与参数；先基线（无复合索引）后优化（恢复索引），各 3 轮

表 2-5 性能测量环境与参数

2.2.3 基线压测原始数据

轮次	吞吐量 (req/s)	请求总数	平均延迟 (ms)	P50 (ms)	P99 (ms)	P97.5 (ms)	2xx / 非 2xx / 错误
1	306.5	4,598	162.30	157	254	236	4,598 / 0 / 0
2	369.5	5,542	134.73	133	171	161	5,542 / 0 / 0
3	355.4	5,331	139.82	132	312	276	5,331 / 0 / 0
平均 / 中位数	343.8	15,471	145.62	133	254	—	15,471 / 0 / 0

表 2-6 基线压测原始数据（50 连接，3 轮 × 15 s）

2.2.4 MongoDB profiler 基线

基线查询使用嵌套的 $and/$or：出发地和目的地各是一个 $or，外层再用 $and 组合，并附加 operatingDates 与 bookable 条件。profiler 共采集 7 次，7 次 executionTimeMillis 分别为 2、3、3、2、3、3、2 ms，中位数 3 ms。

阶段	7 次执行时间 (ms)	中位数 (ms)	执行计划	docsExamined	keysExamined	nReturned
基线	2 / 3 / 3 / 2 / 3 / 3 / 2	3	COLLSCAN	8,000	0	3

表 2-7　基线 profiler 记录

现象：trains 集合只有 _id_ 与 trainNumber_1 索引，查询条件涉及的 departureCity、departureStation、arrivalCity、arrivalStation、operatingDates、bookable 没有可用索引，因此执行计划为 COLLSCAN，检查全部 8,000 条车次文档，最后只返回 3 条。

2.2.5 Clinic Flame 基线

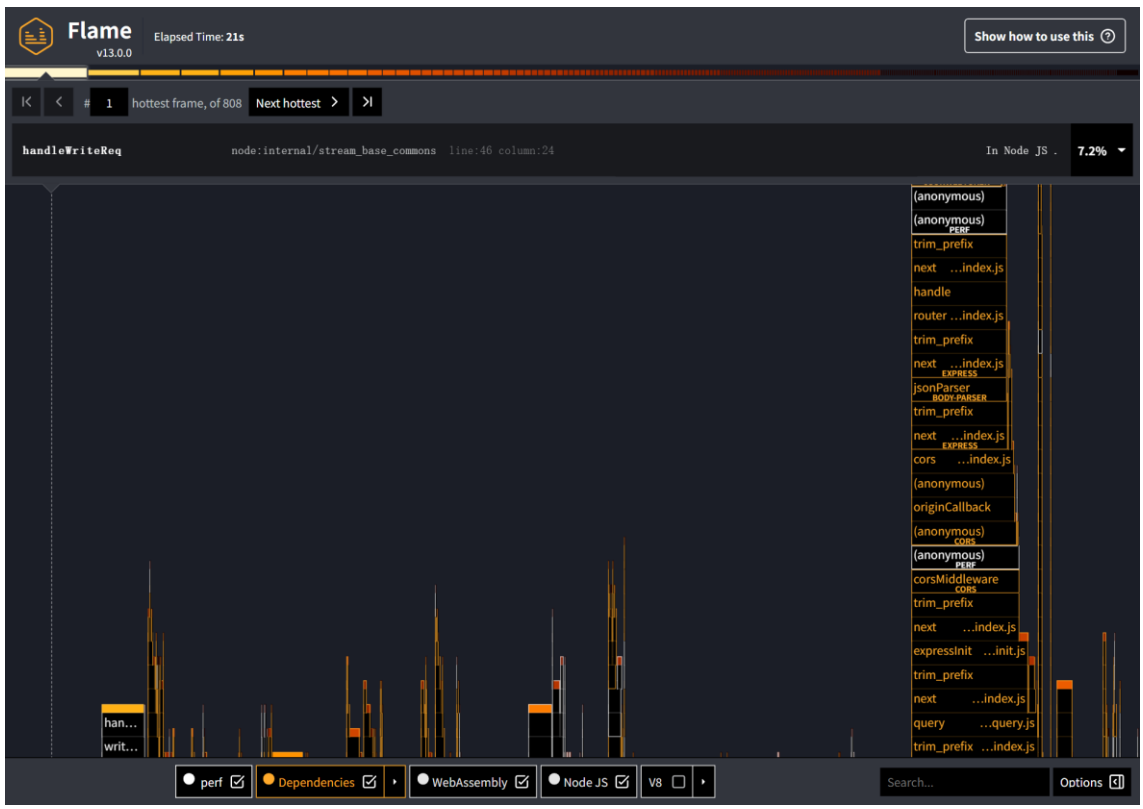


图 2-5　Clinic Flame 基线火焰图

在 30 连接、15 s 的 Clinic 运行中，界面显示 325.4 req/s、P50 87 ms、P99 186 ms；报告共 808 个火焰图节点，默认选中的最热帧为 handleWriteReq（7.2%，Node 流写入路径）。解析内嵌数据可见

createPublicKey 仍有 55 个采样（约 0.38%），HTML 中该文本出现 20 次。

现象：Docker MongoDB 环境下，请求主要受事件循环调度和数据库 I/O 等待限制，createPublicKey 不再是占比最高的单帧；但 JWT 验签在每个请求上仍会先尝试一次注定失败的公钥解析，属于可消除的固定开销。Clinic 会开启采样与插桩，绝对吞吐不能与普通 autocannon 直接比较，因此这里的 325.4 req/s 只用于基线热点分析，不作为最终前后对比口径。

2.3 性能优化实现与源代码对比

本节按文件给出优化前与优化后的源代码。优化前源码快照保存在 experiments/perf/before/，优化后源码快照保存在 experiments/perf/after/，完整 unified diff 保存在 experiments/perf/diff-*.patch。

2.3.1 优化点一：复合索引 + 四路查询

原查询把“出发地”与“目的地”各写成一个 $or，再用 $and 组合。这样的嵌套结构很难让一个复合索引同时覆盖四种起终点组合，MongoDB 最终选择全表扫描。优化方案是显式拆成四种组合：出发城市/出发车站 × 到达城市/到达车站，每个分支都附带 operatingDates 和 bookable 等值条件，并为每种组合建立一条复合索引。

优化前 server/src/controllers/trains.js 全文（18 行）：

```
const { Train, Inventory } = require('../models');
const { plain } = require('../utils/serialize');
const valid = require('../utils/validation');
const { assert } = require('../utils/errors');
async function list(req, res) {
  const date = valid.date(req.query.date);
  const from = valid.text(req.query.from, '出发地');
  const to = valid.text(req.query.to, '目的地');
  assert(from !== to, 400, '出发地与目的地不能相同');
  const trains = await Train.find({$and: [{$or: [{departureCity: from}, {departureStation: from}]}, {$or:
[{arrivalCity: to}, {arrivalStation: to}]}], operatingDates: date, bookable: true}).lean();
  const inventory = await Inventory.find({trainId: {$in: trains.map(t => t._id)}, date}).lean();
  res.json({trains: trains.map(train => ({...plain(train), seats: train.seats.map(seat => ({...seat,
remaining: inventory.find(i => i.trainId === train._id && i.seatType === seat.seatType)?.remaining ??
0}))}))});
}
async function stations(req, res) {
  const trains = await Train.find().select('departureCity arrivalCity departureStation
arrivalStation').lean();
  res.json({stations: [...new Set(trains.flatMap(t => [t.departureCity, t.arrivalCity,
t.departureStation, t.arrivalStation]))].filter(Boolean)});
}
module.exports = { list, stations };
```

代码 2-1　优化前：server/src/controllers/trains.js（全文）

优化后 server/src/controllers/trains.js 全文（27 行）：

```
const { Train, Inventory } = require('../models');
const { plain } = require('../utils/serialize');
const valid = require('../utils/validation');
const { assert } = require('../utils/errors');
async function list(req, res) {
  const date = valid.date(req.query.date);
  const from = valid.text(req.query.from, '出发地');
  const to = valid.text(req.query.to, '目的地');
  assert(from !== to, 400, '出发地与目的地不能相同');
  // 拆成四种起终点组合，每种都能单独命中一条复合索引。
  const routes = [
    {departureCity: from, arrivalCity: to},
    {departureCity: from, arrivalStation: to},
    {departureStation: from, arrivalCity: to},
    {departureStation: from, arrivalStation: to}
  ].map(route => ({...route, operatingDates: date, bookable: true}));
  const trains = await Train.find({$or: routes}).lean();
  const inventory = await Inventory.find({trainId: {$in: trains.map(t => t._id)}, date}).lean();
  // 先按“车次+席别”建索引表，把逐个查找的 O(n×m) 降为 O(n+m)。
  const remaining = new Map(inventory.map(i => [`${i.trainId}|${i.seatType}`, i.remaining]));
  res.json({trains: trains.map(train => ({...plain(train), seats: train.seats.map(seat => ({...seat,
remaining: remaining.get(`${train._id}|${seat.seatType}`) ?? 0}))}))});
}
async function stations(req, res) {
  const trains = await Train.find().select('departureCity arrivalCity departureStation
arrivalStation').lean();
  res.json({stations: [...new Set(trains.flatMap(t => [t.departureCity, t.arrivalCity,
t.departureStation, t.arrivalStation]))].filter(Boolean)});
}
module.exports = { list, stations };
```

代码 2-2　优化后：server/src/controllers/trains.js（全文）

server/src/models/index.js 的 trainSchema 片段对比（完整文件见 before/after 快照）：

优化前（trainSchema 无查询索引）：

```
const trainSchema = new Schema({ ...identity, trainNumber: {type: String, required: true, unique: true}, trainType: String, departureStation: String, arrivalStation: String, departureCity: String, arrivalCity: String, departureTime: String, arrivalTime: String, durationMinutes: Number, operatingDates: [String], stops: [String], seats: [seatSchema], bookable: {type: Boolean, default: true}, revision: {type: Number, default: 0} }, options);
// 每天的库存独立，不能把不同乘车日的余票混用。
// 优化前：trainSchema 之后没有任何 trainSchema.index(...) 定义
```

代码 2-3　优化前：trainSchema 之后没有查询索引

优化后（新增四条复合索引）：

```
const trainSchema = new Schema({ ...identity, trainNumber: {type: String, required: true, unique: true}, trainType: String, departureStation: String, arrivalStation: String, departureCity: String, arrivalCity: String, departureTime: String, arrivalTime: String, durationMinutes: Number, operatingDates: [String], stops: [String], seats: [seatSchema], bookable: {type: Boolean, default: true}, revision: {type: Number, default: 0} }, options);
// 车次查询有“城市/车站”四种起终点组合，各建一条复合索引避免全表扫描。
trainSchema.index({departureCity: 1, arrivalCity: 1, operatingDates: 1, bookable: 1});
trainSchema.index({departureCity: 1, arrivalStation: 1, operatingDates: 1, bookable: 1});
trainSchema.index({departureStation: 1, arrivalCity: 1, operatingDates: 1, bookable: 1});
trainSchema.index({departureStation: 1, arrivalStation: 1, operatingDates: 1, bookable: 1});
```

代码 2-4　优化后：为四种起终点组合各建一条复合索引

索引把 operatingDates 与 bookable 也放入复合索引，目的是在索引扫描阶段就排除不匹配的日期和不可售车次，减少回表读取。operatingDates 是数组字段，MongoDB 会建立 multikey 索引；对数组元素做等值匹配仍可命中索引。

2.3.2　优化点二：余票匹配从线性查找改为 Map

优化前，对每趟车的每个席别都调用 inventory.find(...) 线性扫描库存数组；优化后先按“车次|席别”建立 Map，再 O(1) 查找。核心差异如下：

对比项	优化前	优化后
查找方式	inventory.find(i => i.trainId === train._id && i.seatType === seat.seatType)	remaining.get(`${train._id}\|${seat.seatType}`)
时间复杂度	O(车次数 × 席别数 × 库存数)	O(库存数 + 车次数 × 席别数)
额外空间	无	一个 Map（与库存条数同阶）
适用场景	结果集很小时差异有限	结果集或库存量增大时优势明显

表 2-8　余票匹配优化前后对比

该改动在当前 3 趟车的小结果集上绝对 CPU 收益有限，但它消除的是一个会随结果集增长的二次

复杂度；在真实线路或更大数据量下更有意义。

2.3.3 优化点三：JWT 密钥对象复用

基线火焰图中的 createPublicKey 来自 jsonwebtoken 的验签实现：传入字符串密钥时，它每次都会先尝试把字符串当作非对称公钥解析，抛错后再回退到 HMAC 密钥。把密钥在启动时一次性转换为 KeyObject，即可跳过每个请求上的失败解析。

优化前 server/src/config.js 全文（12 行）：

```
const path = require('node:path');
require('dotenv').config({ path: path.join(__dirname, '../.env') });

// 密钥不能使用默认值启动，防止误部署一个可预测的签名密钥。
function loadConfig() {
  const { MONGO_URI, JWT_SECRET } = process.env;
  if (!MONGO_URI || !JWT_SECRET || JWT_SECRET.length < 32 || JWT_SECRET.startsWith('replace-')) {
    throw new Error('请填写 server/.env：MONGO_URI 和至少 32 字符的随机 JWT_SECRET');
  }
  return { mongoUri: MONGO_URI, jwtSecret: JWT_SECRET, jwtExpiresIn: process.env.JWT_EXPIRES_IN || '8h',
port: Number(process.env.PORT || 3000), origins: (process.env.CORS_ORIGIN ||
'http://localhost:5500').split(',').map(x => x.trim()) };
}
module.exports = { loadConfig };
```

代码 2-5 优化前：server/src/config.js（全文）

优化后 server/src/config.js 全文（14 行）：

```
const path = require('node:path');
const { createSecretKey } = require('node:crypto');
require('dotenv').config({ path: path.join(__dirname, '../.env') });

// 密钥不能使用默认值启动，防止误部署一个可预测的签名密钥。
function loadConfig() {
  const { MONGO_URI, JWT_SECRET } = process.env;
  if (!MONGO_URI || !JWT_SECRET || JWT_SECRET.length < 32 || JWT_SECRET.startsWith('replace-')) {
    throw new Error('请填写 server/.env：MONGO_URI 和至少 32 字符的随机 JWT_SECRET');
  }
  // 先转成 KeyObject；否则 jsonwebtoken 每次验签都会尝试 createPublicKey 再回退到 HMAC 密钥。
  return { mongoUri: MONGO_URI, jwtSecret: createSecretKey(Buffer.from(JWT_SECRET)), jwtExpiresIn:
process.env.JWT_EXPIRES_IN || '8h', port: Number(process.env.PORT || 3000), origins:
(process.env.CORS_ORIGIN || 'http://localhost:5500').split(',').map(x => x.trim()) };
}
module.exports = { loadConfig };
```

代码 2-6　优化后：server/src/config.js（全文）

兼容性验证：用字符串密钥签发的旧 token，仍可被同样字节的 KeyObject 验证；优化后的火焰图 HTML 中 createPublicKey 出现 0 次（基线为 20 次）。该改动只改变密钥对象的创建时机，不改变签名算法、密钥字节或 token 格式。

2.3.4　代码改动清单

文件	改动内容	优化前	优化后	证据
server/src/controllers/trains.js	四路 $or 查询；Map 余票匹配	18 行	27 行	代码 2-1 / 2-2；diff-trains.patch
server/src/models/index.js	新增 4 条车次复合索引	34 行	39 行	代码 2-3 / 2-4；diff-models.patch
server/src/config.js	JWT 密钥转为 KeyObject	12 行	14 行	代码 2-5 / 2-6；diff-config.patch
experiments/perf/server.js	测试服务器按阶段选择字符串/KeyObject 密钥，并显式传入测试密钥	测试设施	测试设施	after/server.js（不计入业务代码改动）

表 2-9　代码改动清单

2.4　优化后复测

2.4.1 优化后压测原始数据

轮次	吞吐量 (req/s)	请求总数	平均延迟 (ms)	P50 (ms)	P99 (ms)	P97.5 (ms)	2xx / 非 2xx / 错误
1	400.8	6,012	123.88	119	248	157	6,012 / 0 / 0
2	418.6	6,279	119.03	114	196	179	6,279 / 0 / 0
3	452.6	6,789	110.24	108	154	137	6,789 / 0 / 0
平均 / 中位数	424.0	19,080	117.72	114	196	—	19,080 / 0 / 0

表 2-10　优化后压测原始数据（50 连接，3 轮 × 15 s）

2.4.2 MongoDB profiler 复测

阶段	7 次执行时间 (ms)	中位数 (ms)	执行计划	docsExamined	keysExamined	nReturned
基线	2 / 3 / 3 / 2 / 3 / 3 / 2	3	COLLSCAN	8,000	0	3
优化后	0 / 0 / 0 / 0 / 0 / 0 / 0	0	4 路 IXSCAN	3	6	3

表 2-11　优化前后 profiler 对比

profiler 的 executionTimeMillis 为整数毫秒，优化后 7 次均为 0，表示低于 1 ms 计时精度，不表示绝对零耗时。优化后的 planSummary 明确包含四条复合索引的 IXSCAN：

```
IXSCAN { departureCity: 1, arrivalCity: 1, operatingDates: 1, bookable: 1 }
IXSCAN { departureCity: 1, arrivalStation: 1, operatingDates: 1, bookable: 1 }
IXSCAN { departureStation: 1, arrivalCity: 1, operatingDates: 1, bookable: 1 }
IXSCAN { departureStation: 1, arrivalStation: 1, operatingDates: 1, bookable: 1 }
```

代码 2-7　优化后 profiler 中的四条 IXSCAN

2.4.3 Clinic Flame 复测

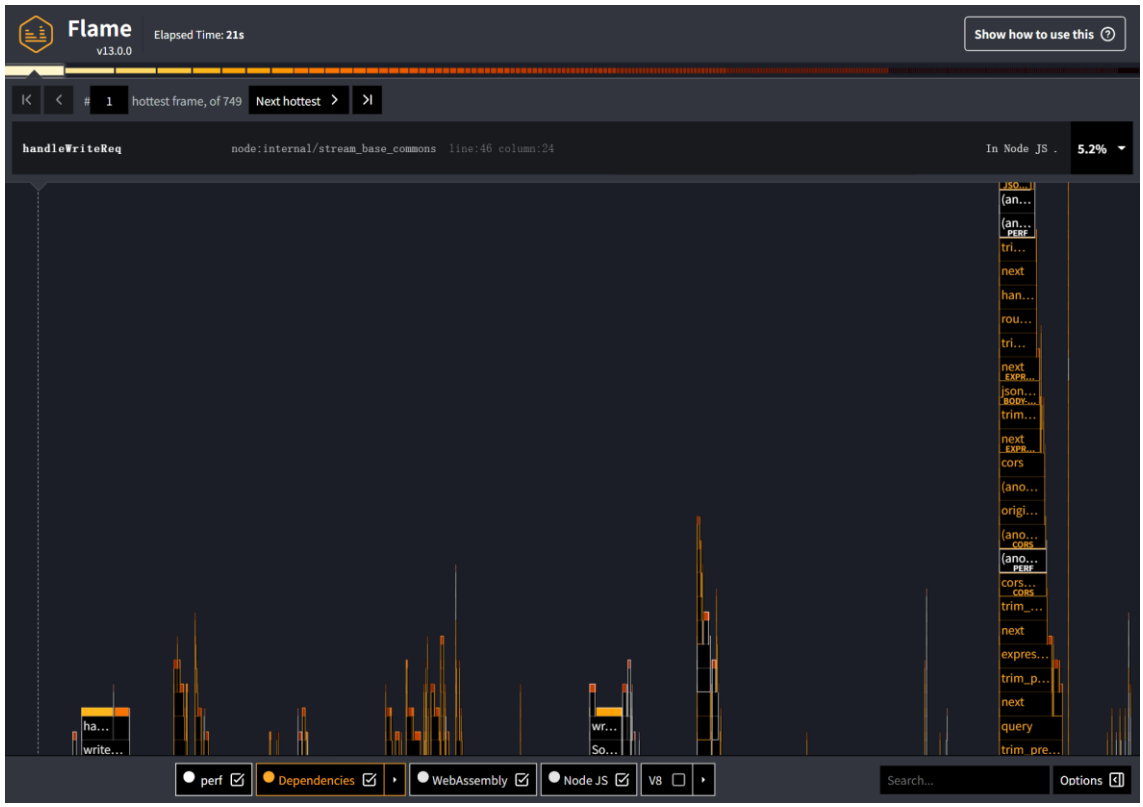


图 2-6　Clinic Flame 优化后火焰图

优化后 Clinic 报告共 749 个火焰图节点，默认选中的最热帧仍为 handleWriteReq（5.2%，HTTP/流写入路径）；createPublicKey 在优化后报告中出现 0 次，说明 JWT 密钥构造热点已消除。采样中剩余的顶层帧主要是 processTicksAndRejections、AsyncFunctionAwaitResolveClosure、mongoose writeCommand/exec 等 I/O 等待路径，说明请求已更多受数据库/网络等待限制，而不是 CPU 计算。

本次 Clinic 优化后运行达到 430.8 req/s、P50 66 ms、P99 159 ms，高于基线那一次运行；但 Clinic 会开启采样与插桩，绝对吞吐仍不能与普通 autocannon 直接比较，因此前后吞吐以 autocannon 配对测量为准，Clinic 数据主要用于确认热点是否存在。

2.4.4　功能正确性与回归测试

- 同一查询返回 3 趟车：G1001、G1002、G1003；席别与余票保持为商务座 40、一等座 100、二等座 500。
- 压测期间所有响应均为 2xx；基线 3 轮合计 15,471 个 2xx，优化后 3 轮合计 19,080 个 2xx，非 2xx 与错误均为 0。
- npm --prefix server test：35 个测试全部通过，覆盖下单、支付、退票、改签、权限、会话过期、索引迁移等回归场景。

2.5 实验过程发现的问题及处理

序号	问题现象	原因判断	处理方式
1	安装 eslint-plugin-no-unsanitized 时 ERESOLVE	插件 4.x 要求 ESLint 9，项目使用 ESLint 8.57.1	移除该插件，XSS 由 Semgrep 覆盖
2	ESLint HTML 报告默认折叠，截图看不到问题详情	报告中的表格行未展开	截图前用 Puppeteer 点击所有 th 展开
3	Semgrep 的 extra.lines 返回 requires login	报告渲染器尝试访问在线源码	改为从本地源文件读取对应行
4	npm audit 在 npmmirror 上返回 404	镜像未实现 advisories/bulk 接口	改用官方 registry 重新审计
5	MongoDB explain 读取时报 BSONError: Invalid UTF-8 string	驱动/explain 返回结构与解析路径不兼容	改用 profiler level 2，直接读取 system.profile
6	Clinic 首次运行时停在遥测确认	非交互终端无法确认	设置 NO_INSIGHT=1 跳过
7	Clinic Doctor 在 Node 24 上报 No files matching the pattern found	node-trace-log-join 与当前 Node 版本不兼容	改用 Clinic Flame
8	Clinic --autocannon 带鉴权头时打印用法	参数解析不接受该组合	改用 --on-port 执行程序化 autocannon 脚本
9	Clinic --on-port 未加 -- 时打印帮助	Clinic 把后面的 node server.js 当成多余参数	在 --on-port 脚本后补 --，再写 node experiments/perf/server.js
10	Puppeteer 对优化后 Clinic HTML 使用 networkidle0 超时	页面持续有采样数据更新，网络空闲条件难以满足	Clinic 页面改用 waitUntil: load，并等待 5 s
11	早期非 Docker 环境连续压测后吞吐从 400 左右降到 123–227 req/s	机器级 I/O 状态漂移，而非代码异常；所有请求仍为 2xx	改用 Docker 隔离 MongoDB 并重新建库配对测量；早期数据归档到 archive-pre-docker-2026-10-22
12	Docker CLI 报 docker-credential-desktop 不在 PATH	只调用了 docker.exe 全路径，凭据助手目录未加入 PATH	把 C:\Program Files\Docker\Docker\resources\bin 加入 PATH 后重试
13	拉取 mongo:7.0.14 时 auth.docker.io 返回 EOF	首次访问 Docker Hub 的网络握手失败	重试后拉取成功，镜像摘要核对一致
14	P99 单轮波动（基线 171–312 ms，优化后 154–248 ms）	尾部延迟受机器抖动影响大，3 轮样本不足以稳定估计	报告 3 轮原始数据与中位数，不把单轮极值当作稳定结论

表 2-12　实验过程中遇到的问题与处理

2.6 实验复现命令

静态审查命令已见 2.1.1。性能实验的复现步骤如下（在项目根目录执行）：

```
# 1. 启动隔离 MongoDB（Docker Desktop，Linux 容器）
docker run -d --name mini12306-perf-mongo \
  -p 127.0.0.1:27017:27017 \
  -v mini12306-perf-mongo-data:/data/db \
  mongo:7.0.14 mongod --replSet rs0 --bind_ip_all
docker exec mini12306-perf-mongo mongosh --quiet \
  --eval 'rs.initiate({_id:"rs0",members:[{_id:0,host:"127.0.0.1:27017"}]})'
# .mongo-uri 写入：
# mongodb://127.0.0.1:27017/mini12306_perf?replicaSet=rs0

# 2. 基线：恢复 experiments/perf/before/ 的三个快照后导入数据
node experiments/perf/seed-large.js

# 3. 登录测试用户并写入 .token
node experiments/perf/get-token.js

# 4. 基线：另开终端启动 3100 端口，采集执行计划并压测
PERF_PHASE=baseline node experiments/perf/server.js
node experiments/perf/explain.js baseline
node experiments/perf/load.js baseline

# 5. 恢复 after/ 与优化代码，启动优化服务后复测
PERF_PHASE=optimized node experiments/perf/server.js
node experiments/perf/explain.js optimized
node experiments/perf/load.js optimized

# 6. 汇总指标
node experiments/perf/summarize.js
```

代码 2-8　性能实验复现命令

Clinic Flame 基线示例：

```
NO_INSIGHT=1 PERF_PHASE=baseline DURATION=15 CONNECTIONS=30 \
  experiments/tools/node_modules/.bin/clinic flame \
  --dest experiments/perf/clinic-before-docker --name baseline \
  --on-port "node experiments/perf/load-on-port.js" \
  -- node experiments/perf/server.js
```

代码 2-9　Clinic Flame 运行命令

2.7 第二轮补充优化（选做，独立测量）

第二轮是独立于实验二的补充优化：独立建库、独立对照，不与实验二的 343.8 → 424.0 req/s 数据合并或相加。实验二仍以车次查询优化为主体结论；本节只处理另外两处可测量的问题：站点接口的整表重复读取，以及订单过期清理的全表扫描与重复执行。

测量环境：Docker mongo:7.0.14 单节点副本集 rs0，127.0.0.1:27019；数据库 mini12306_perf_round2；8,000 趟车、20,000 条订单，测试账号名下 20 条订单；50 连接，3 s 预热 + 3 轮 × 15 s。测试服务通过 experiments/perf/round2/server.js 启动，显式传入测试 JWT 密钥，不读取 server/.env。before 变体只回退本轮三项改动，after 直接使用当前 server/src。

2.7.1 站点接口缓存与源代码对比

基线问题：GET /api/trains/stations 每次请求都执行 Train.find() 读取全部车次，把 8,000 条文档的四个站点字段物化到 Node 后再去重。这些站点是公共数据，50 并发时每个请求都在重复同一次全表读取；实测只有 6.7 req/s，P50 为 5,160 ms。

优化实现：新增 server/src/utils/station-cache.js，在进程内缓存公共站点 5 分钟。命中时返回数组副本；冷缓存的并发请求共享同一次 inFlight 查询；generation 计数保证缓存失效前启动的旧查询不能回填；读取失败不缓存，下一次请求可以重试。管理员新增、修改、删除车次的事务提交后调用 invalidateStationCache()，无需等待 TTL。缓存只保存公共站点，不保存用户、会话或订单数据。

优化前 controllers/trains.js 的站点函数（实验二结束时的代码）：

```
async function stations(req, res) {
  const trains = await Train.find().select('departureCity arrivalCity departureStation
arrivalStation').lean();
  res.json({stations: [...new Set(trains.flatMap(t => [t.departureCity, t.arrivalCity,
t.departureStation, t.arrivalStation]))].filter(Boolean)});
}
```

代码 2-10 优化前：stations 每次请求全表读取并去重

优化后 controllers/trains.js 的对应部分：

```
const { getStations } = require('../utils/station-cache');
...
async function stations(req, res) {
  res.json({stations: await getStations()});
}
```

代码 2-11 优化后：stations 委托给进程内缓存

```
const STATION_CACHE_TTL_MS = 5 * 60 * 1000;
let cachedStations = null;
let expiresAt = 0;
let generation = 0;
let inFlight = null;

function invalidateStationCache() {
  generation += 1;
  cachedStations = null;
  expiresAt = 0;
  inFlight = null;
}

function getStations() {
  if (cachedStations !== null && Date.now() < expiresAt) {
    return Promise.resolve([...cachedStations]);
  }
  if (inFlight) return inFlight.promise.then(stations => [...stations]);

  const startedGeneration = generation;
  const flight = {};
  flight.promise = Promise.resolve().then(async () => {
    const trains = await Train.find().select('departureCity arrivalCity departureStation arrivalStation').lean();
    const stations = [...new Set(trains.flatMap(train => [train.departureCity, train.arrivalCity,
train.departureStation, train.arrivalStation]))].filter(Boolean);
    if (generation === startedGeneration) {
      cachedStations = stations;
      expiresAt = Date.now() + STATION_CACHE_TTL_MS;
    }
    return stations;
  }).finally(() => {
    if (inFlight === flight) inFlight = null;
  });
  inFlight = flight;
  return flight.promise.then(stations => [...stations]);
}
```

代码 2-12　优化后：station-cache.js 的缓存核心逻辑

2.7.2　订单过期索引与源代码对比

基线问题：GET /api/orders 会先调用 expireOrders()，查询 status = 'pending' 且 expiresAt ≤ 当前时间的订单。订单集合有 20,000 条，原有索引只有 userId+status 和 userId+requestId，没有匹配该查询的索引，执行计划为 COLLSCAN，扫描 20,000 条文档，中位数 9 ms。

优化实现：在 orderSchema 上新增 {status: 1, expiresAt: 1} 复合索引，保留原有用户订单索引与唯一约束。查询同时包含 status 等值和 expiresAt 范围，字段顺序与条件匹配，未过期的 pending 订单不会进入索引范围。该索引用于查询，不是 TTL 删除索引；订单取消仍走原事务与条件更新，不会绕过状态机直接删除订单。

优化前 models/index.js 的订单索引：

```
orderSchema.index({userId: 1, status: 1});
orderSchema.index({userId: 1, requestId: 1}, {unique: true, partialFilterExpression: {requestId: {$type:
'string'}}});
```

代码 2-13 优化前：没有匹配过期查询的索引

优化后 models/index.js 的订单索引：

```
orderSchema.index({userId: 1, status: 1});
orderSchema.index({status: 1, expiresAt: 1});
orderSchema.index({userId: 1, requestId: 1}, {unique: true, partialFilterExpression: {requestId: {$type:
'string'}}});
```

代码 2-14 优化后：新增 status + expiresAt 复合索引

2.7.3 过期清理 single-flight 与源代码对比

基线问题：GET /api/orders 的每个调用者都会独立执行一轮 expireOrders()。同一时刻 10 个并发请求会执行 10 次相同的过期扫描；虽然条件更新能保证库存不重复释放，但重复扫描会额外占用数据库和事件循环。

优化实现：模块级 expirationFlight 保存正在执行的清理 Promise，并发调用者共享同一轮；结束后在 finally 中清空，失败会向等待者传播，下一轮仍可重试。每张订单的查询条件、事务边界和库存释放逻辑不变。

优化前 utils/order-service.js 的入口：

```
async function expireOrders() {
  return runExpiration();
}
```

代码 2-15 优化前：每个调用者独立执行清理

```
let expirationFlight = null;
function expireOrders() {
  if (!expirationFlight) {
    expirationFlight = runExpiration().finally(() => { expirationFlight = null; });
  }
  return expirationFlight;
}
```

代码 2-16　优化后：utils/order-service.js 的 single-flight 入口

2.7.4　改动清单与验证

文件	本轮改动	规模 / 证据
server/src/controllers/trains.js	stations 改为调用 getStations()	删除 2 行内联查询；新增 1 行 import 与 1 行调用；代码 2-10 / 2-11
server/src/utils/station-cache.js	新增 5 分钟 TTL、冷请求合并、generation 防护	43 行；代码 2-12
server/src/controllers/admin.js	新增、修改、删除车次后失效站点缓存	3 处 invalidateStationCache()
server/src/models/index.js	新增订单 status + expiresAt 复合索引	39 → 41 行；代码 2-13 / 2-14
server/src/utils/order-service.js	expireOrders 增加 single-flight	入口 3 行 → 7 行；代码 2-15 / 2-16

表 2-13　第二轮代码改动清单

验证：npm --prefix server test 共运行 47 个测试，47 通过、0 失败、0 跳过，退出码 0。测试覆盖缓存首次读取与命中、去重、响应副本隔离、TTL 边界、显式失效、冷缓存并发合并、旧查询不回填、读取失败不缓存、订单索引存在性、single-flight 共享与失败重试、管理员真实 API 写入后立即失效等场景。测试耗时是套件耗时，不是接口性能数据。

复现与原始数据：脚本位于 experiments/perf/round2/，原始 JSON 与 profiler/explain 结果位于 experiments/perf/round2/results/，汇总为 round2-summary.json；完整方法见 experiments/第二轮性能测量.md。测量容器在结束后停止，数据卷保留。

第三部分　结果与讨论（可加页）实验结果分析（包括数据处理、实验现象分析、影响因素讨论、综合分析和结论等）

3.1　数据处理

压测的吞吐量取 3 轮算术平均值；P50、P99 取 3 轮的中位数，避免某一轮抖动主导结论；数据库层指标取 7 次 profiler 采样的中位数。百分比变化统一按（优化后 − 基线）/ 基线 × 100% 计算。

指标	基线	优化后	计算式	变化
吞吐量	343.8 req/s	424.0 req/s	(424.0 − 343.8) / 343.8	+23.33% ≈ +23.3%
平均延迟	145.62 ms	117.72 ms	(117.72 − 145.62) / 145.62	−19.16% ≈ −19.2%
P50 延迟	133 ms	114 ms	(114 − 133) / 133	−14.29% ≈ −14.3%
P99 延迟	254 ms	196 ms	(196 − 254) / 254	−22.83% ≈ −22.8%
扫描文档数	8,000	3	(3 − 8000) / 8000	−99.96%
查询执行时间	3 ms	0 ms（<1 ms）	(0 − 3) / 3	−100%（低于计时精度）

表 3-1　性能指标变化计算

一致性校验：在固定 50 并发下，按 Little 定律，吞吐量应近似等于并发数 / 平均延迟。基线 50 / 0.14562 s ≈ 343.4 req/s，实测 343.8 req/s（相差约 0.1%）；优化后 50 / 0.11772 s ≈ 424.7 req/s，实测 424.0 req/s（相差约 0.2%）。两组数据相互吻合，说明压测结果内部一致，吞吐提升主要来自平均延迟下降，而不是请求计数错误。

3.2　实验结果分析

3.2.1　静态审查结果分析

工具	版本	审查范围	结果	人工判断
ESLint（前端）	8.57.1 + security 3.0.1	js/**/*.js	24 个问题（15 error / 9 warning）	无阻断发布的错误；以不可见空白、块内函数声明和启发式告警为主
ESLint（后端）	8.57.1 + security 3.0.1	server/src/**/*.js	15 个 warning，0 error	无语法/潜在运行时错误；对象注入告警需结合调用方确认
Semgrep	1.182.0	项目源码（p/nodejs + p/owasp-top-ten）	4 条 raw-html-concat WARNING	人工复核为误报：所有插值均经过 escape()；但 innerHTML 写法仍有回归风险
npm audit	npm 11.16.0	server 依赖树	149 个依赖，0 个漏洞	依赖层未发现已知 CVE；不覆盖业务代码

表 3-2　静态审查结果汇总

三个工具的结论互相补充：ESLint 给出了可维护性问题清单；Semgrep 的 4 条 XSS 告警经数据流复核为误报；npm audit 未发现依赖漏洞。综合判断，本次审查没有发现需要立即修复的高危问题，也没有修改任何业务代码。

需要避免两个极端：一是把工具告警数量直接等同于风险数量，例如 7 条 security/detect-object-injection 和 4 条 raw-html-concat 都需要结合代码上下文判断；二是因为 npm audit 为 0 就认为系统安全，它只覆盖依赖的已知漏洞，不检查业务逻辑和访问控制。

3.2.2 数据库层优化分析

指标	基线	优化后	变化
执行计划	COLLSCAN	4 路 IXSCAN	消除全表扫描
docsExamined	8,000	3	−99.96%
keysExamined	0	6	由无索引扫描变为索引扫描
nReturned	3	3	结果一致
执行时间（中位数）	3 ms	0 ms（<1 ms）	−100%（低于计时精度）

表 3-3 数据库层指标对比

优化后每次查询只检查 3 条文档、6 个索引键，说明索引前缀与查询条件匹配良好：四种起终点组合各自命中一条复合索引，operatingDates 与 bookable 在索引层完成过滤，最终回表文档数与 nReturned 一致。查询时间从 3 ms 降到 0 ms（profiler 的计时精度为 1 ms，即低于 1 ms），也验证了执行计划的改变确实生效，而不只是表面上的 planSummary 变化。

索引的代价是写入时维护 4 条 B-tree 以及额外磁盘占用。车次数据属于读多写少的配置型数据，当前取舍合理；如果未来车次写入频率显著上升，应结合写入延迟和磁盘成本重新评估索引数量。

3.2.3 端到端性能分析

指标	基线	优化后	变化	结论
吞吐量	343.8 req/s	424.0 req/s	+23.3%	稳定提升
平均延迟	145.62 ms	117.72 ms	−19.2%	稳定下降
P50 延迟	133 ms	114 ms	−14.3%	稳定下降
P99 延迟	254 ms	196 ms	−22.8%	中位数改善，单轮仍有波动
2xx 总数	15,471	19,080	+23.3%	与吞吐提升一致
非 2xx / 错误	0 / 0	0 / 0	—	无错误

表 3-4 端到端性能指标对比

为什么数据库扫描文档数减少了 99.96%，端到端吞吐提升 23.3%？该接口每个请求还要执行 JWT 验签、User.findById、Session.findOne、Inventory.find 以及 HTTP/JSON 处理。按 Amdahl 定律，单段加速对整体性能的提升受该段在总耗时中占比的限制。

可以做一次量级估算：基线的车次查询中位时间为 3 ms，平均请求延迟约 145.62 ms，前者约占 2.1%；即使把 3 ms 降到 1 ms 以下，理论上只减少不到 2 ms/请求，约 1.4% 的平均延迟。实测平均延迟下降 27.9 ms（19.2%），高于这一估算，说明 JWT KeyObject 与 Map 也消除了部分固定开销，且测量本身存在波动；三项优化是叠加起作用的，不能把全部收益单独归因于索引。

P99 的变化需要谨慎解读：基线三轮为 254/171/312 ms，优化后为 248/196/154 ms，中位数从 254 ms 降到 196 ms。优化后三轮均不高于基线中位数，但基线单轮最低为 171 ms，且样本只有 3 轮，仍不能把 P99 视为稳定改善；吞吐、平均延迟和 P50 在三轮中均一致向好，是更可靠的结论。

3.2.4 功能正确性分析

优化不能以牺牲正确性为代价。复测确认同一查询仍返回 G1001、G1002、G1003 三趟车，席别余票仍为 40、100、500；压测期间 15,471/19,080 个请求全部为 2xx，非 2xx 与错误均为 0；35 个自动化测试全部通过。说明查询改写、Map 余票匹配和 KeyObject 替换都没有改变接口语义。

3.3 影响因素讨论

- 环境漂移：早期非 Docker 环境有一轮基线为 392.8/432.5/390.2 req/s，连续压测后同一机器的测量值一度降到约 123–227 req/s，但所有请求仍为 2xx。为避免把机器状态变化误判为代码效果，最终改用 Docker 隔离 MongoDB，在同一容器、同一份数据上做基线/优化配对测量，早期数据归档而不混入最终结论。
- 容器 I/O：Docker Desktop 的 Linux 容器通过 WSL2 访问数据卷，I/O 路径与本机进程不同；基线/优化在同一容器内完成，相对比较有效，但绝对吞吐不能与 bare-metal 直接比较。
- Amdahl 定律：车次查询只是请求链路的一段。鉴权、用户查询、会话查询、库存查询和 HTTP 处理仍然存在，因此数据库层的巨大改善不会等比例转化为端到端吞吐。基线车次查询中位数 3 ms，仅占平均延迟约 2.1%；若要继续提升，应优先减少鉴权链路的数据库往返，但这涉及会话失效语义，本次没有贸然改动。
- Clinic 采样开销：Clinic 会插桩并采样，绝对吞吐不能与普通 autocannon 直接比较；本次基线 325.4 req/s、优化后 430.8 req/s 只用于热点定性比较。Clinic 的有效信号是 createPublicKey 从 20 次降为 0 次，而不是它的吞吐数字。
- P99 与样本量：尾部延迟对机器抖动、GC 和后台进程敏感，基线单轮 P99 在 171–312 ms 之间波动，优化后为 154–248 ms；3 轮 × 15 s 的样本不足以稳定估计 P99，后续可增加轮次、采用 A-B-A 交替设计并报告置信区间。
- 数据分布：8,000 趟车、168,000 条库存是合成数据，查询固定为武汉→北京；真实线路的日期分布、热门线路集中度和缓存命中率不同，索引收益会变化，绝对数值不能直接外推到生产。
- 索引成本：4 条复合索引会增加车次写入时的索引维护和磁盘占用。当前是读多写少的路径，收益大于成本；若写入频率上升，需要重新评估。
- KeyObject 类型变化：config.jwtSecret 从字符串变为 KeyObject。当前 jwt.sign/jwt.verify 都支持该类型，35 个测试通过；若项目外还有代码把该字段当字符串使用，需要同步调整。
- 静态工具局限：ESLint 的 security 规则和 Semgrep 的 raw-html-concat 都是模式匹配，不能替代

人工数据流分析；npm audit 只覆盖已知依赖漏洞，不检查业务逻辑和权限控制。

3.4 综合分析与结论

静态审查方面，三个工具覆盖了代码规范、语义安全和依赖漏洞三个层面。ESLint 报告前端 24 个问题（15 error、9 warning）和后端 15 个 warning，主要是不可见空白、块内函数声明、对象注入启发式告警和未使用变量；Semgrep 的 4 条 XSS 告警经 escape() 数据流复核为误报；npm audit 为 0 漏洞。没有发现需要立即修复的高危问题，实验一未修改任何业务代码。

性能优化方面，完成了三项改动：四条复合索引 + 四路查询把车次查询从 COLLSCAN 改为 4 路 IXSCAN，docsExamined 从 8,000 降到 3（−99.96%），查询时间从 3 ms 降到 0 ms（低于 1 ms 计时精度）；Map 把余票匹配从 O(n×m) 降为 O(n+m)；KeyObject 消除了 createPublicKey 热点（HTML 中出现次数 20 → 0）。

端到端配对压测结果为：吞吐 343.8 → 424.0 req/s（+23.3%），平均延迟 145.62 → 117.72 ms（−19.2%），P50 133 → 114 ms（−14.3%），P99 254 → 196 ms（−22.8%，单轮仍有波动）。15,471/19,080 个请求全部为 2xx，35/35 自动化测试通过，接口返回车次和余票一致。

工程启示是：优化前必须先测量，优先处理占比最大的路径；数据库扫描优化非常显著，但端到端收益会被鉴权、库存查询和 HTTP 开销稀释。报告同时保留数据库层和端到端两组指标，避免用局部数字代表整体，也避免把环境漂移误写成优化效果。

3.5 后续改进方向

- 鉴权链路：评估合并 User.findById 与 Session.findOne，或在保证会话失效语义的前提下做短 TTL 缓存，减少每请求的数据库往返。
- 测量方法：采用 A-B-A 交替多轮、增加每轮时长，报告均值、中位数与置信区间；在独立压测机上复测，减少本机工具与服务争抢 CPU 的影响。
- 数据规模：用更接近真实线路分布的合成数据（热门线路集中、日期跨度更大）验证索引选择性，并观察不同查询的 planSummary。
- 代码质量：单独提交清理不可见空白、块内函数声明和未使用变量；评估把 innerHTML 模板改为 DOM API/textContent，降低 XSS 回归风险。
- 持续性能：把关键查询的 docsExamined、P50、P99 纳入回归测试或监控，避免后续改动重新引入全表扫描。

3.6 第二轮补充优化结果分析

第二轮的所有负载请求均为 2xx，无非 2xx、错误或超时。吞吐与延迟取 3 轮中位数，数据库指标取 profiler/explain 的中位数；下列数字只在本轮 before/after 之间可比，不能与实验二的 343.8 → 424.0 req/s 相加。

数据口径：站点接口按 50 连接、3 s 预热 + 3 轮 × 15 s 取中位数，另做 20 次顺序请求和一次冷缓存采样；订单接口同样 3 轮，executionStats 使用 7 条已过期加 3 条未过期 pending 订单；数据库

工作量用 profiler level 2 逐请求统计；single-flight 每次同时发起 10 个请求，重复 3 轮，每轮重置 20 条过期订单。所有轮次均记录 2xx、非 2xx、错误和超时。

3.6.1 站点接口缓存结果

指标	before（每次读库）	after（缓存）	变化
吞吐量（50 连接）	6.7 req/s	300.0 req/s	+4,377.6%（44.8 倍）
平均延迟	6,233.66 ms	166.03 ms	−97.34%
P50 延迟	5,160 ms	163 ms	−96.84%
P99 延迟	8,296 ms	428 ms	−94.84%
2xx / 非 2xx / 错误	100 / 0 / 0	4,500 / 0 / 0	无错误

表 3-5 站点接口 50 并发结果（3 轮中位数）

44.8 倍是“8,000 条车次 + 50 并发”的特定对照值：before 的每个请求都重复全表读取，并发越高放大越明显。更保守的是 20 次顺序请求：中位延迟 62.56 → 15.12 ms（−75.83%，约 4.1 倍），P95 72.21 → 16.63 ms。冷缓存首个请求 231.14 → 255.09 ms，没有变快；首次仍要读库，缓存只消除后续重复读取。

指标	before	after	说明
20 次顺序请求中位延迟	62.56 ms	15.12 ms	−75.83%
顺序请求 P95	72.21 ms	16.63 ms	−76.97%
冷缓存首个请求	231.14 ms	255.09 ms	单次采样，无提升
热缓存 10 次请求的 trains 查询	10 次	0 次	profiler 未捕获
每请求扫描文档数	8,000	冷 8,000 / 热 0	before 另有 8 次 getMore

表 3-6 站点接口顺序请求与数据库工作量

before 每个请求执行 1 次 trains 查询和 8 次 getMore，共扫描 8,000 条文档；after 冷缓存仍扫描一次，热缓存的 10 次连续请求中没有 trains 查询。命中路径仍要执行 JWT 校验、用户查询和会话查询，所以 300 req/s 与 15 ms 不是“零成本”，只是移除了公共站点的整表读取。

3.6.2 订单索引与 single-flight 结果

指标	无索引	有索引	变化
执行计划	COLLSCAN	IXSCAN status_1_expiresAt_1	消除全表扫描
docsExamined	20,000	7	−99.97%
keysExamined	0	7	进入索引范围
nReturned	7	7	结果一致
执行时间（中位数）	9 ms	0 ms（<1 ms）	−100%（低于计时精度）

表 3-7 订单过期查询执行统计（7 条已过期 +3 条未过期 pending）

7 条已过期订单全部命中，3 条未过期订单不进入索引范围；回表文档数与返回数一致。GET /api/orders 的 50 并发结果如下：

指标	before	after	变化
吞吐量	125.6 req/s	136.5 req/s	+8.68%
平均延迟	393.7 ms	363.89 ms	−7.57%
P50 延迟	398 ms	357 ms	−10.30%
P99 延迟	778 ms	650 ms	−16.45%
错误数	0	0	—

表 3-8　订单列表接口 50 并发结果（3 轮中位数）

订单接口只提升 8.68%，远小于扫描文档数的下降幅度。原因是该接口还要执行 JWT 校验、用户/会话查询、订单列表查询和 JSON 序列化；索引只消除了过期查询这一段。这个百分比不能写成通用吞吐提升，也不能与实验二的 +23.3% 相加。逐请求 profiler 显示，过期查询的扫描量从每请求 20,000 条降到 0 条：首轮清理后已无过期 pending 订单，后续查询不再需要回表。

指标	无 single-flight	有 single-flight	变化
过期扫描次数（3 轮）	10 / 10 / 10	1 / 1 / 1	−90%
取消订单数（每轮）	20	20	一致
HTTP 200 请求数（每轮）	10	10	一致
墙钟中位数	3,395.64 ms	3,216.86 ms	−5.26%（波动大）

表 3-9　single-flight 并发清理结果（10 个并发请求、20 条过期订单）

single-flight 的确定收益是把同一时刻 10 个调用者的过期扫描合并为 1 次；墙钟范围（无合并 3,216.32–3,946.79 ms，有合并 1,375.35–3,635.54 ms）重叠，−5.26% 不能视为稳定延迟收益。20 条订单逐条事务仍是主要耗时，合并不会减少事务数量。

3.6.3　正确性、边界与结论

- 正确性：第二轮压测请求全部为 2xx，没有 non-2xx、错误或超时；47/47 自动化测试通过；每轮 20 条过期订单都只取消一次，没有重复释放库存；订单原索引与唯一约束保留。
- 缓存语义：热缓存返回数组副本；读取失败不缓存；管理员新增、修改、删除车次的事务提交后立即失效，其他直接写库或跨进程写入最多等待 5 分钟 TTL。冷缓存首个请求没有变快，缓存消除的是后续请求的重复整表读取。
- 单进程边界：缓存与 single-flight 都只在一个 Node.js 进程内有效，多实例各自维护一份，没有共享缓存或分布式锁；数据库条件更新和事务仍是防止重复释放的最终保护。
- 数量级边界：44.8 倍只对应“8,000 条车次 + 50 并发”的重复全表读取场景；顺序请求约 4.1 倍是更保守的观察。单节点 Docker、合成数据和 20 条过期订单积压不等于真实生产负载，绝对数值不能外推。

- 独立性：第二轮是独立测量，不修改实验二的参数、数字和结论；实验二的车次查询优化仍是主体优化，本节数据不能与 343.8 → 424.0 req/s 相加，也不能把 +8.68% 写成通用吞吐提升。

小结：三项改动中，站点缓存收益最大，直接消除了热路径上的整表读取；订单索引把单次过期查询从 20,000 条文档降到 7 条；single-flight 把并发过期扫描从 10 次降到 1 次，但没有测到稳定的墙钟收益。订单接口只提升 8.68%，再次说明数据库层的局部改善会被鉴权、查询和序列化等固定开销稀释，必须与端到端指标一起报告。

成绩评定表：

序号	评分项目	满分	实得分
1	实验报告格式规范	2	
2	实验报告过程清晰，内容详实	4	
3	实验报告结果正确性	2	
4	实验分析与总结详尽	2	
	总得分	10	